

# Scaling with Network Namespaces in a Single Application

PJ Waskiewicz
Principal Engineer
October 6th, 2016
Netdev 1.2, Tokyo, Japan


© 2016 NetApp, Inc. All rights reserved.
NetApp

# Agenda

1) Why use network namespaces?
2) Application architecture: how do namespaces fit?
3) Application architecture: lifecycle management
   - Namespace creation, identification, and switching
4) Efficient switching between namespaces
5) Scalability concerns
6) Proposed improvements


© 2016 NetApp, Inc. All rights reserved.

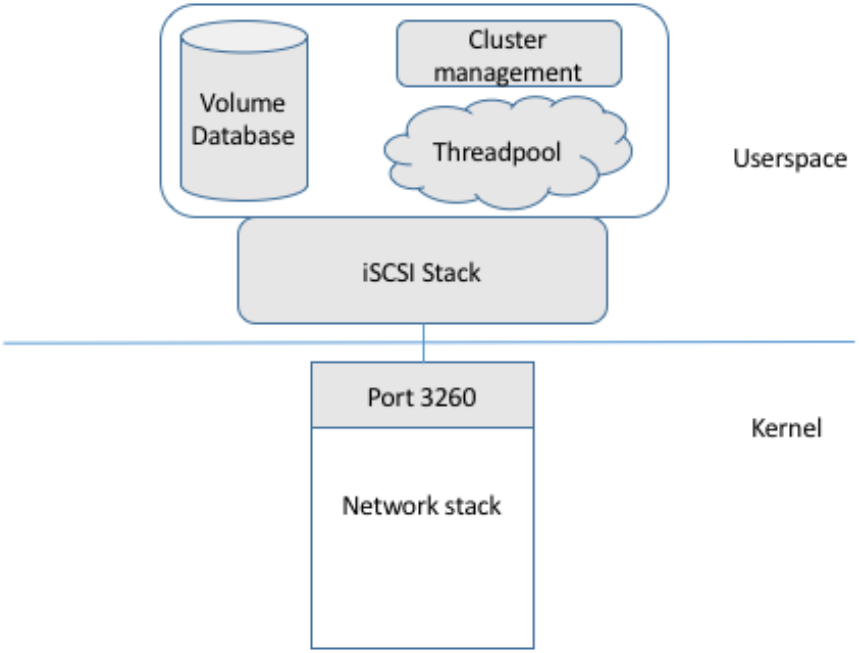
# Why use network namespaces?

- Cloud-based computing and infrastructure continues to grow
- Many existing applications can be put inside a container
  - Can either be run inside a VM, or using LXC or Docker
- Some existing applications will need pieces of themselves containerized
  - Core logic may support multi-tenancy, but abstracted from OS layers
  - Networking layer of application may need to place each tenant on independent L3 stacks
  - Make an application essentially VRF-aware


© 2016 NetApp, Inc. All rights reserved.

# Application architecture: how do namespaces fit?

- Simple application overview:

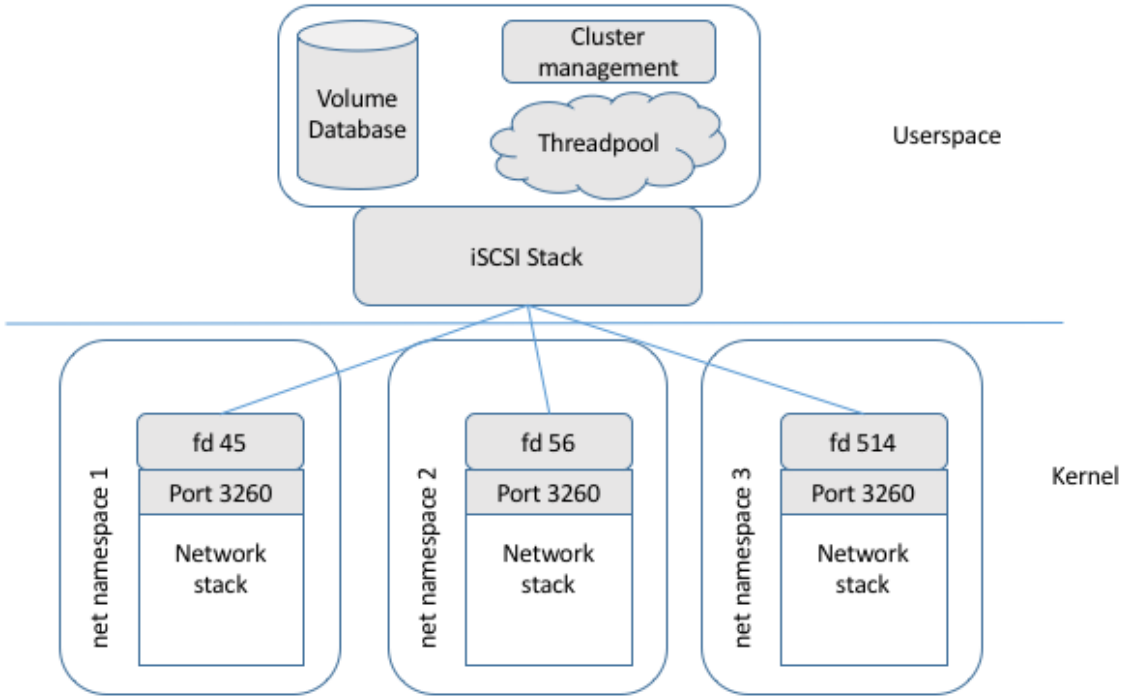




© 2016 NetApp, Inc. All rights reserved.
NetApp

# Application architecture: how namespaces fit (cont.)

- Application uses file descriptor for each TCP port bound inside of network namespace





© 2016 NetApp, Inc. All rights reserved.
NetApp

# Application architecture: lifecycle management

- Network namespaces can be created multiple ways:
  - From the command line: `ip netns add <namespace name>`
  - Using netlink directly (issue same commands that `ip netns` will send)
  - Using `clone()` with the `CLONE_NEWNET` flag
- Programmatically, creating via command-line is most-straightforward
- Once namespace has been added, now accessible in `/run/netns`
  - New special file is created with the name of the namespace
  - File can be opened, and file descriptor is used by `setns()` to switch into that namespace
- Any threads or applications bound to that namespace can also access through `/proc`
  - `/proc/<pid>/ns/net` is same file in `/run/netns`


© 2016 NetApp, Inc. All rights reserved.

# Application architecture: lifecycle management (cont)

- Identifying a namespace by name is not enough to manage multiple namespaces
- Namespaces can be created and destroyed often, depending on application needs
  - File descriptor from `/run/netns/<namespace name>` may be same integer, but different namespace. Non-deterministic.
- Using `mtime` from `stat()` on `/run/netns/<namespace name>` provides pretty fine granularity
- Storing both a file descriptor to `/run/netns/<namespace name>` plus the `mtime` of that file allows application to keep track of namespaces


© 2016 NetApp, Inc. All rights reserved.

# Application architecture: lifecycle management (cont)

- Switching threads between namespaces within an application is done with `setns()` system call

```
int ns_fd = open("/run/netns/namespace_1", O_RDONLY);
int rc = setns(ns_fd, CLONE_NEWNET);
```

- May need to return to default/base namespace
  - Need a way to get back to original namespace network stack

```
int base_fd = open("/proc/1/ns/net", O_RDONLY);
int rc = setns(base_fd, CLONE_NEWNET);
```

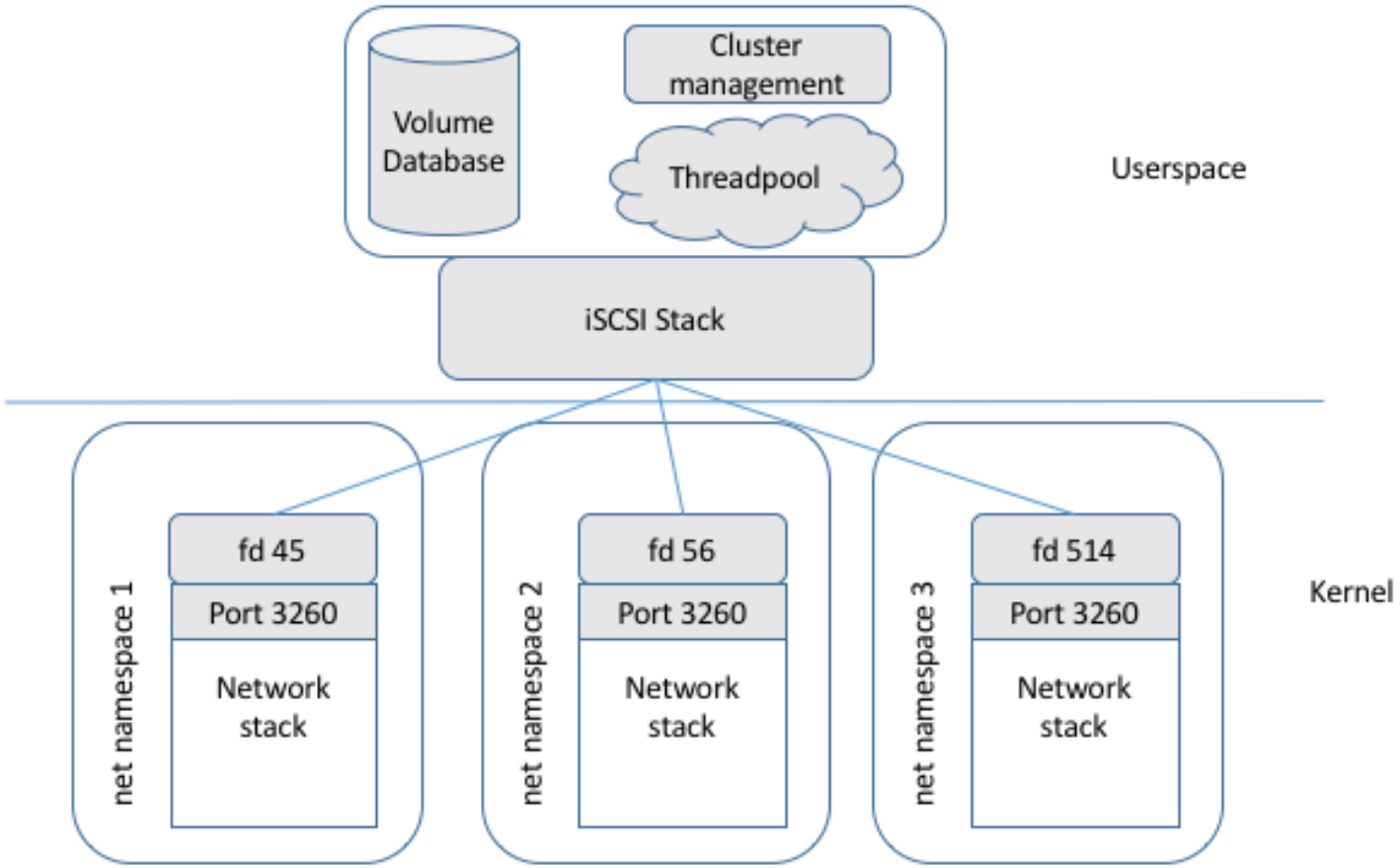

© 2016 NetApp, Inc. All rights reserved.

# Efficient namespace switching

- Not desirable to have worker threads always switching namespaces
  - `setns()` system call overhead can get expensive
  - More namespaces with lots of threads can get messy
- Using stored file descriptor and `mtime` of a namespace helps identify namespaces
- Each thread can store same metadata of current namespace inside thread-local storage
- If scheduled thread is already in the namespace needing processing, avoid the `setns()` call


© 2016 NetApp, Inc. All rights reserved.

# Efficient namespace switching (cont)





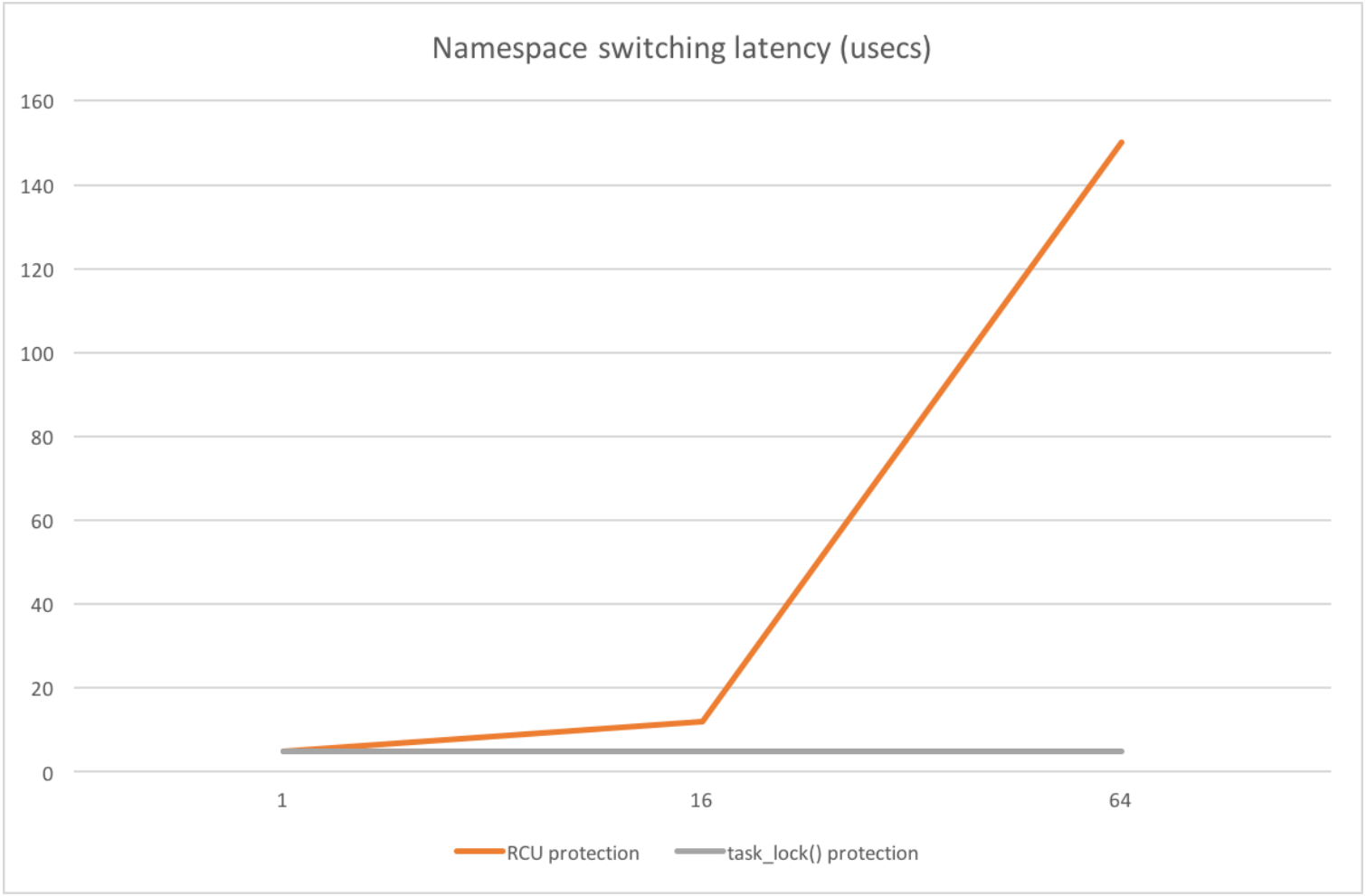
© 2016 NetApp, Inc. All rights reserved.
NetApp

# Scalability concerns

- Earlier versions of nsproxy handling in the kernel presented big problems with scalability
- Adding more network namespaces caused more switching of threads, leading to very large spikes in latency
- Fixed by Eric Biederman in later 3.x kernels
- Highlights need for focus on changes in this area of the kernel


© 2016 NetApp, Inc. All rights reserved.

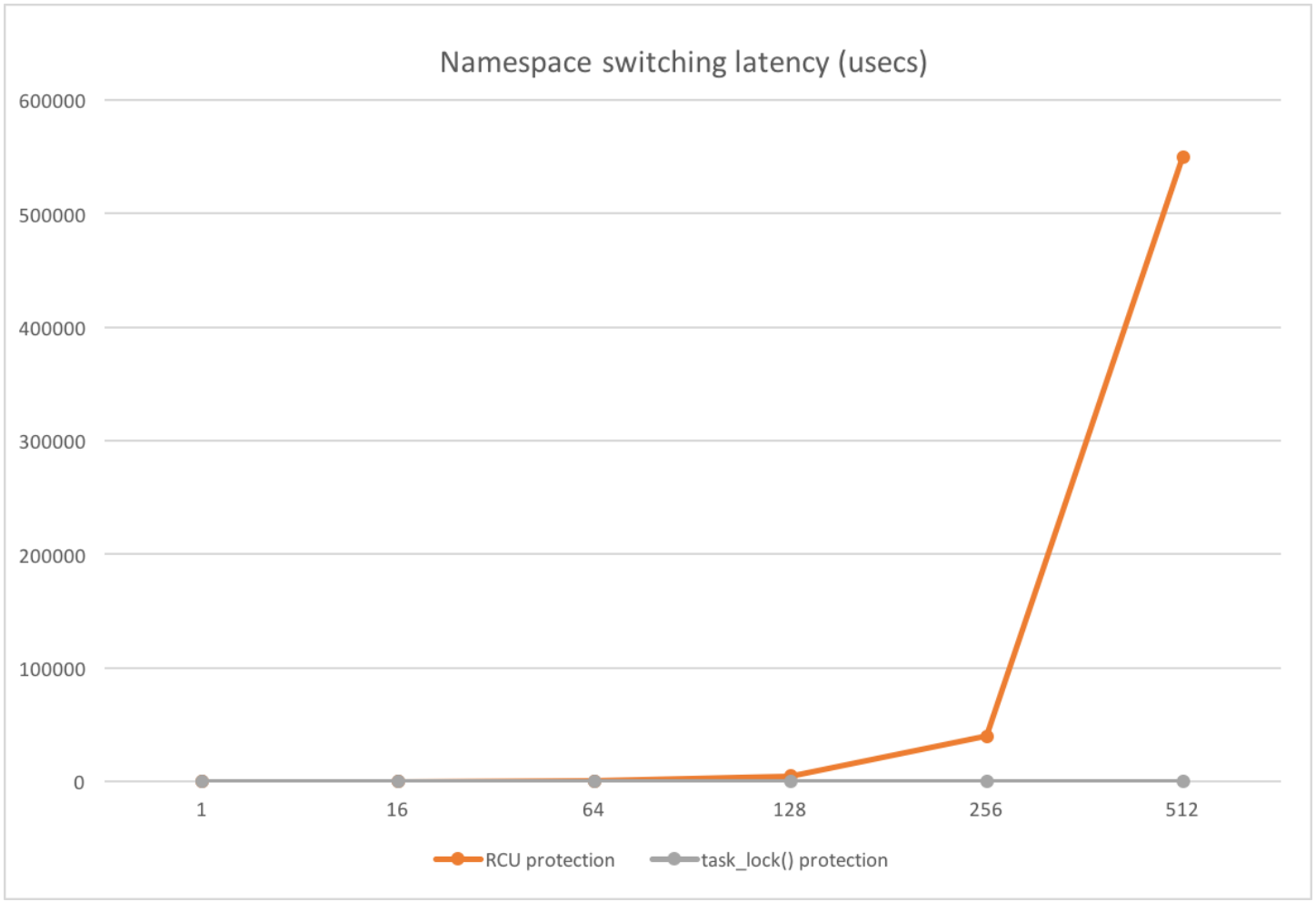
# Scalability concerns (cont)





© 2016 NetApp, Inc. All rights reserved.
NetApp

# Scalability concerns (cont)





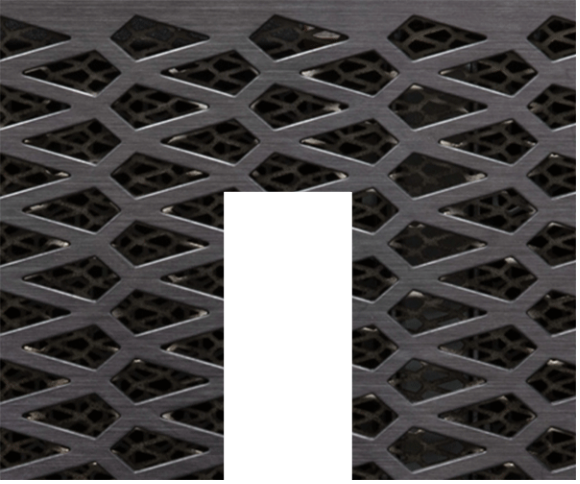
© 2016 NetApp, Inc. All rights reserved.
NetApp

# Proposed improvements

- Add torture tests/unit tests to Kernel Hacking to look for performance regressions of namespace switching
  - Similar to RCU torture tests, but for latency detection

- Need better ”unique” identification mechanisms
  - `mtime` plus file descriptor works, but basing on time can cause unexpected problems
  - Clock skews can cause misidentification of a namespace
  - Propose a UUID-like mechanism, exposed perhaps through `/proc/<pid>/ns`, or other interface TBD
  - With UUID-like mechanism, uniqueness is virtually guaranteed; designed to be unique


© 2016 NetApp, Inc. All rights reserved.



# Thank you.


© 2016 NetApp, Inc. All rights reserved.

NetApp